Conclusion DLISP Lab 3

Carlos Celaya, Marcel Hofmann

1 Training PSPNet without Class Weighting

1.1 Nearest Neighbor Interpolation Explanation

In semantic segmentation tasks, the resize operation applied to ground-truth label maps must preserve the discrete class values. Nearest neighbors is the simplest method to interpolate and assigns the value of the nearest known data point to an unknown location.

Alternative interpolation strategies such as bilinear or bicubic interpolation compute pixel values as weighted averages of their neighborhood. While this is desirable for natural images, it is catastrophic for label masks. It produces blurry class transitions, like for the color changes in Figure 1. Nearest Neighbor preserves the labels exactly, keeping sharp transitions between regions and ensuring that each pixel corresponds to a valid class index.

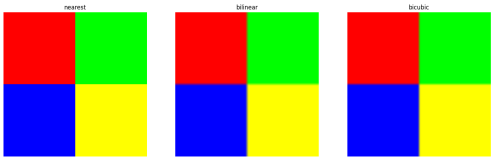


Figure 1: This shows a resized image from 50x50 to 200x200 using different interpolation methods named above them. Nearest Neighbor interpolation does conserve strict color borders (wanted for semantic segmentation). The other methods create blurry transitions.

1.2 Accuracy and Loss for Baseline Experiment

Figure 2a shows the evolution of test accuracy and Figure 2b the loss (train and test) for the simplified PSPNet with a pretrained ResNet18 backbone.

The accuracy stabilizes around 0.6, which is below what is typically expected for VOC-like datasets. However, accuracy is not an ideal metric for semantic segmentation because the VOC dataset is highly imbalanced (most pixels belong to "background"). So, a model can reach high accuracy by predicting mainly the majority class.

The loss curves show clear overfitting, with training loss decreasing while validation loss stagnates or increases. This behavior indicates that the model is memorizing dominant classes rather than learning generalizable patterns.

This motivates the need for better metrics, such as per-class IoU, which is introduced below.

1.3 IoU Analysis

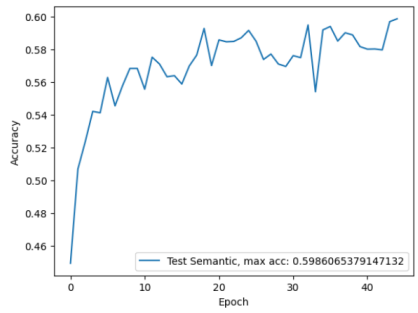
Figure 2c reports the IoU values for each semantic class. The model demonstrates strong class imbalance effects: Classes with many training pixels (background, car, train) reach the highest IoU. Rare classes with only a few pixels (bicycle, chair, sofa) achieve extremely low IoU, showing that the network struggles to learn under-represented categories.

Surprisingly, the model does not collapse completely into predicting only background, even though VOC has some imbalance. However, the disparity between classes reflects that frequency-based dominance is still the largest factor affecting performance.

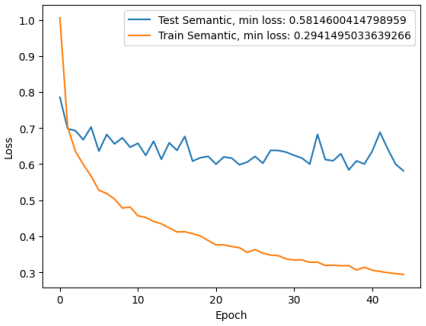
1.4 Hyperparam Tuning and Experiments

Achieving a baseline accuracy of around 60% is already quite decent as baseline. Simple hyperparameter changes such as learning rate and milestones can already influence the model's performance. The problem during training is that the model is strongly overfit to the data it sees. Too early learning rate updates would lead to too fast convergence since update steps are not big enough anymore.

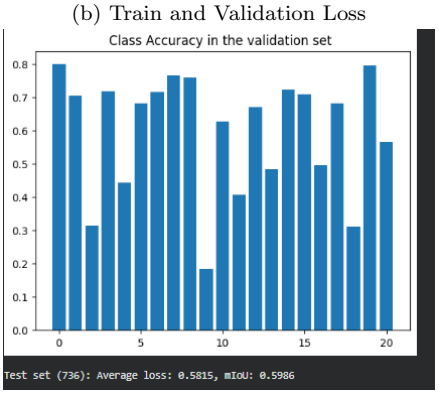
Such hyperparameter changes need to be combined with other techniques such as data augmentation. We implemented an approach to rescale the image via a random factor, then do the random scrop with preserved image dimensions (224x224) to make them fit into the network. Color jitter is applied as well to increase



(a) Validation Accuracy (mIoU)



(b) Train and Validation Loss



(c) IoU per class validation

Figure 2: Plots for the baseline experiment (45 epochs)

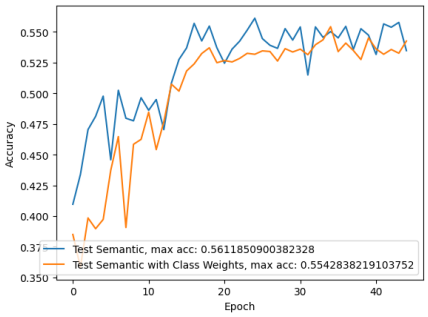
diversity even more. There are also other methods that increase data diversity without being harmful for semantic segmentation, e. g. small rotations or using Gaussian blurring filters (to make segmentation extremely hard because of blurry borders.

The results of such a training approach setting milestones at epoch 15, 25 and 35 and starting with learning rate 0.01 (a bit higher as usual) can be seen in Figure. 3. The usage of the milestones lead to a longer convergence. Still, results are worse than in the baseline. This is because because of the appliance of data augmentation making it harder to train the model in general. More epochs would be needed and probably also a higher share of epochs with higher learning rates, since these epochs are the ones that effect the model performance the most.

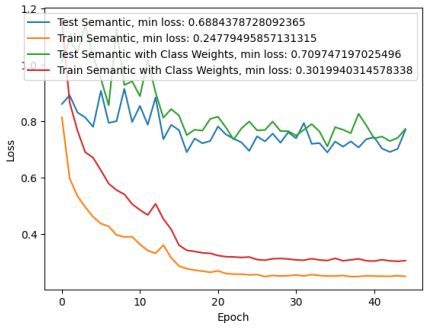
A similar approach can be used for a comparison to training from scratch, setting the learning rate at the start a bit higher to let the model explore the loss hyperplane more effectively and overcome possible local minima. Still, results are worse than using a pretrained model.

The results of this training can be seen in Figure 4. Also here, for further experiments, the milestones should be set later because after some milestones, convergence is inevitable. The total accuracy is worse than the total accuracy for the pretrained model. Still, there are some easier classes that achieve accuracies around 50%. Pretrained models benefit from pretrained kernels already ideal for detecting simply edges or more class related patterns. Randomly initialized models need to learn them still. This is a hard task when having highly diverse images.

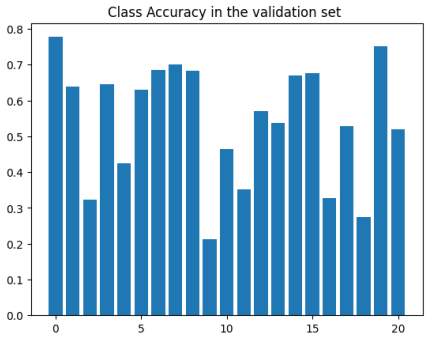
Another important factor that may improve training is using higher resolution images. But this also increases training time



(a) Validation Accuracy (mIoU)

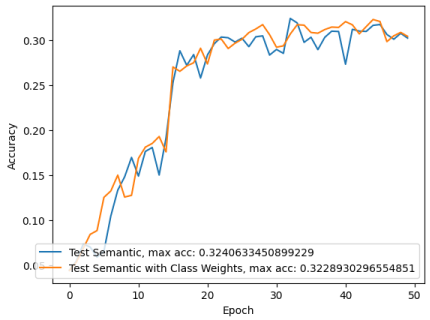


(b) Train and Validation Loss

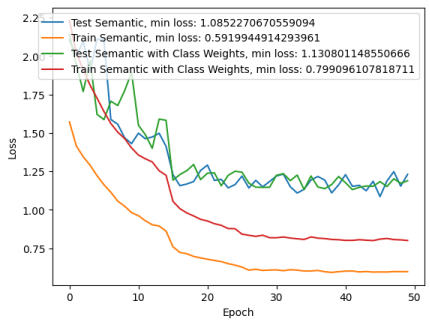


(c) IoU per class validation

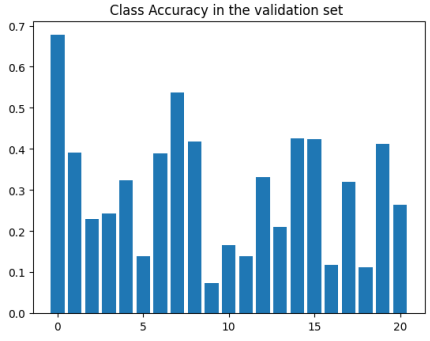
Figure 3: Plots for training pretrained model with data augmentation and milestones.



(a) Validation Accuracy (mIoU)



(b) Train and Validation Loss



(c) IoU per class validation

Figure 4: Plots for training model from scratch with data augmentation and milestones.

strongly because the model needs bigger linear layers that can cope with higher input dimensions. In our example, images are resized and cropped to 224x224 (and thus made smaller than original) to match the dimensions of the pretrained model. However, making images smaller removes (maybe relevant) details from it that could improve the segmentation process.

2 Training PSPNet-like using Class Weighting

2.1 Accuracy and Loss for Class Weighted Baseline

Figure 5a shows the accuracy and Figure 5b the losses when using a class-weighted cross entropy, where weights are inversely proportional to class frequency. The figures also contain the curves of the non-class weighted variant. Interestingly, the difference between the balanced and unbalanced models is small. This suggests that the imbalance is too strong for simple inverse-frequency weighting.

2.2 IoU Comparisons

Figure 5c confirms that class-weighting yields only marginal improvements. Some minority classes improve slightly, but the overall distribution remains similar. This indicates that reweighting alone is insufficient, probably because the background class still dominates. So future iterations should explore the focal loss, adaptive reweighting, or ignoring background pixels during training.

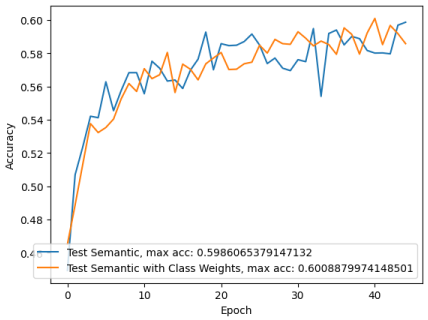
2.3 Visual comparisons

As it can be seen in the figure 6 the model segmentates the low level features or the basic information about where and what shape an object has and the information of the center of an object. However the information about the details and the high level structure is failed by both models and it shows clearly that in some cases like the plain the weight balanced performs even more poorly than the other one which is consistent with the accuracies and loss graphs during training and test. Also it cann be seent hat it classifies highly better the majority class (trains and planes) rather than the minority class (chairs or sofas). This demands an increase on the with rebalancing techniques.

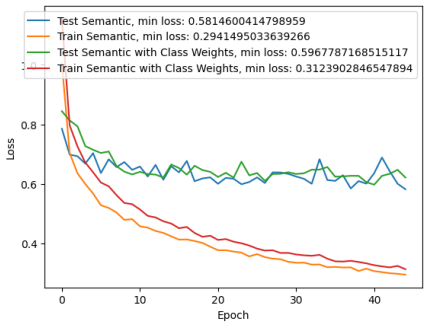
2.4 Hyperparam Tuning and Experiments

All experiments were conceived as alternative strategies to mitigate the strong class imbalance of the dataset by adjusting the loss behaviour, modifying the sampling distribution, or increasing the effective exposure of minority classes during training. The learning curves and per-class IoU plots referenced throughout this section can be reproduced by executing the provided training scripts.

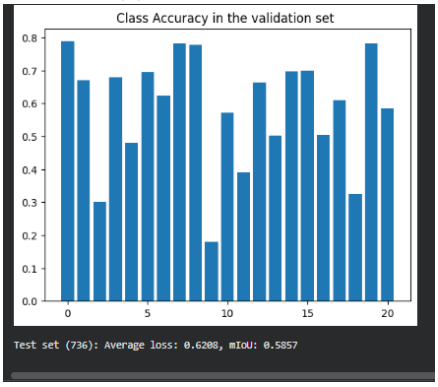
Experiment 1: α-scaled inverse-frequency reweighting. This experiment applied a modified inverse-frequency weighting in which each class weight was raised to a controllable exponent α, normalized so that the minimum weight equalled 1. The aim was to apply a stronger yet stable emphasis on minority classes due to the insufficiency of the baseline rebalancing scheme. A relatively aggressive value of $\alpha = 0.65$ was tested to amplify the contribution of underrepresented classes while avoiding excessively large gradients. Although the training loss decreased faster, validation accuracy and validation loss were degraded and significantly more unstable, and the IoU of the target minority classes (2, 9, 18) did not improve. This behaviour reflects an excessive amplification of noisy, scarce minority pixels: the model overfit to these limited patterns rather than learning generalizable features.



(a) Acc. (mIoU) Comparison



(b) Loss Comparison



(c) Class weighted val IoU

Figure 5: Plots describing the baseline class weighted training and also comparing it to non class weighted training

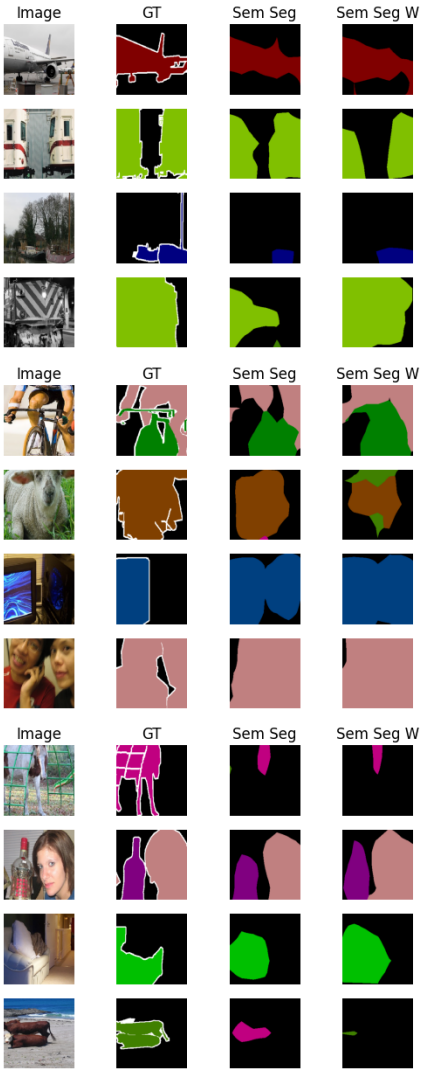


Figure 6: Visual comparisons between the PSPNet model and the other class weighted model.

Experiment 2: Effective-number reweighting. Class weights were instead computed using the "effective number of samples", a formulation intended to provide a smoother and theoretically grounded reweighting than naive inverse-frequency. Two values of β close to one were considered. With $\beta = 1 - 10^{-7}$ the effective numbers changed only marginally compared with the baseline, producing negligible differences in performance. With $\beta = 1 - 10^{-8}$ the class weights increased sharply for the rarest classes and performance deteriorated. These outcomes illustrate the extreme sensitivity of the method in highly imbalanced settings: if β is not tuned to a very narrow window, reweighting is either too weak to matter or so strong that it destabilizes training by amplifying noise rather than meaningful minority information.

Experiment 3: Online Hard Example Mining (OHEM). The loss was computed per pixel, but only the hardest pixels (those with the largest per-pixel loss) contributed to the optimisation step. It was expected that emphasising difficult regions would indirectly highlight minority classes without hand-crafted class weights. The training and validation losses became slightly closer, suggesting a small reduction in overfitting, yet accuracy and per-class IoUs did not improve. In practice, many of the selected "hard" pixels corresponded to ambiguous background borders rather than minority objects, so the intended increase in attention to minority classes did not materialise.

Experiment 4: CurriculumSampler. A curriculum sampling strategy was implemented by gradually increasing the fraction of images containing minority pixels over the course of training. The objective was to allow the model to first organise the feature space using easier examples and progressively introduce harder ones, especially those with minority classes. The learning dynamics changed, showing smoother behaviour during early epochs, but after the curriculum phase ended and the full dataset was introduced, the final performance largely matched the baseline. Changing the order in which information is presented improves convergence but not the ultimate capacity to learn rare classes when the data itself remains insufficiently diverse.

Experiment 5: Targeted augmentation via smart-crop oversampling. Images containing minority pixels were cropped around those regions and then resized back to the input resolution, aiming to increase the effective frequency and apparent diversity of minority samples without altering the loss function. Contrary to expectations, performance worsened: accuracy dropped and per-class IoUs declined. The cropping introduced scale and context shifts relative to the original images, reducing the compatibility with the pretrained backbone and harming generalisation. Moreover, repeatedly cropping from the same limited minority regions did not add real semantic variety, leading to overfitting on narrowly defined patterns.

Across all experiments a consistent pattern emerged. Approaches targeting rebalancing (via α-scaling or effective numbers) or refocusing of the training signal (via OHEM, curriculum sampling, or targeted augmentation) altered gradient contributions but did not provide stable or significant benefits for minority classes. These methods successfully reduced the dominance of background gradients but typically increased gradient variance, amplified annotation noise, or disrupted contextual cues necessary

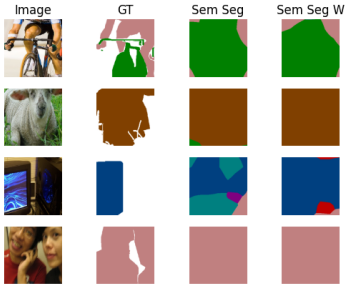


Figure 7: Ignoring the background leads to accidental misclassifications of it.

for generalisation. Consequently, minority-class IoUs did not exhibit consistent improvements and in several cases declined.

Two practical conclusions follow. First, reweighting and mining techniques should be applied conservatively and accompanied by stronger regularisation (e.g. dropout, weight decay, or mixup-like strategies for segmentation) together with richer augmentations so that any amplified gradients correspond to informative rather than noisy signals. Second, data-centric interventions are essential: increasing the diversity of minority-class examples, generating realistic synthetic data, or combining class reweighting with loss functions designed to handle frequency imbalance (e.g. hybrids of Dice/IoU with cross-entropy or class-dependent focal variants). Ignoring the background class during loss computation (using an *ignore_index*) can better align training with IoU-based objectives, but on its own does not address the lack of intra-class diversity required for reliable improvements.

2.5 Ignoring Background

There is the possibility to ignore the background during training by setting the background pixels to the ignored, non penalized pixels during training. Usually, this class is for difficult border regions (e.g. cow lattice fence in Figure 6) that cannot be classified well. When also assigning the background to this class, the model never learns to classify the background well, leading to accidental misclassifications. Segmentation masks tend to "bleed" into the ignored regions during the segmentation. The effect can be nicely seen in Figure 7: Regions of an object flow into the background region as well because these regions were ignored during the loss calculation. This is because such predictions are not penalized during training. During such trainings, general pixel accuracy may be low, but metrics not focusing on background (e.g. mIoU) improve which can be misleading. Such an approach is only helpful when the misclassifications of the really background do not matter and only a vague localization of object structures is important.

3 Qualitative evaluation of pretrained models

In this section, we compare the behavior of the models by first describing their general characteristics and then analyzing their performance on the initial image.

ResNet-50 and ResNet-101 are convolutional neural networks based on deep residual learning, where the use of skip connections allows them to learn increasingly complex hierarchical features without suffering from vanishing gradients; the main difference between them lies in depth, with ResNet-101 capturing more detailed and fine-grained information thanks to its additional layers. DeepLabv3, on the other hand, is a semantic segmentation architecture that relies on atrous convolutions and an Atrous Spatial Pyramid Pooling (ASPP) module, enabling it to achieve multi-scale contextual understanding and to refine object boundaries more effectively

Both ResNet-50 and ResNet-101 capture the overall structure of the scene effectively (figure 8), showing that they understand where major objects such as the horse or the person are located; however, they also introduce spurious classification patches corresponding to objects that are not actually present. Between the two, ResNet-101 offers slightly better performance, as it is able to identify smaller details such as the woman's clamp. In



Figure 8: These images represent the original image of a horse and the segmentations made by the ResNet50 architecture and the ResNet101 respectively. The segmentation is blurry but visible.



Figure 9: These images are the result of the segmentation of the original image of a horse, made by the Deeplabv3 ResNet50 architecture and the Deeplabv3 ResNet101 respectively. The segmentaiton is very similar to the previous ones.

contrast, DeepLabv3 performs substantially better by producing cleaner predictions without hallucinated regions (figure 9), and both of its evaluated backbone configurations achieve consistently solid segmentation results. The time usage of each architecure corresponds to the expectations as the Deeplabv3 is more complex and the number of layers also increase the amount of complexity. The other image chosen can be seen in the figure 11. This image was chosen because it represents several objects at a long distance from the camera. The segmentation here is more challenging than the previous one. Similar results were obtained for all the architectures as the figures 11 and 12 shows,the vast majority of the objects were segmented, however it is very blurry. Which demonstrates that for long distance objects shorter architectures can give the same results as the architectures with less resources.

3.1 Effect of changing image resolutions

Different scales of images lead to a loss of information when an image is made smaller (scale below 1) and a gain of information when it is made bigger (scale above 1). How traits of an image change with different scales can be observed in Figure 13. Usually, images are scaled down, as in our semantic segmentation project.

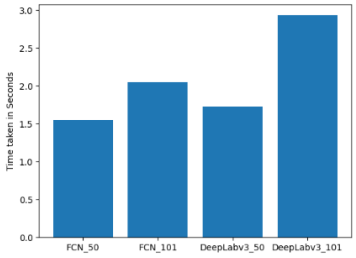


Figure 10: Time spent by each algorithm to segment the original image. As the model increase in complexity also does the time.



Figure 11: These images represent the original image of a horse and the segmentations made by the ResNet50 architecture and the ResNet101 respectively. Noise is very visible in both images

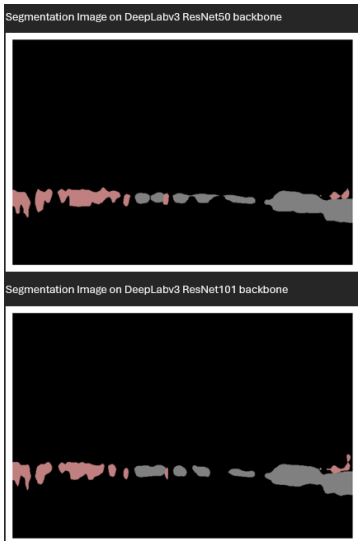


Figure 12: These images are the result of the segmentation of the original image of a horse, made by the Deeplabv3 ResNet50 architecture and the Deeplabv3 ResNet101 respectively. Both segmentations are very good.



Figure 13: Comparison of images at different scales (the values in the brackets). The smaller the scale, the more details get lost, making the images more blurry.

Since pixels are taken away from objects, they receive less space to be present in an image, eventually leading to disappearing smaller structures (e. g. single hairs of the photo in Figure 13). Also, from object borders, pixels are taken away which leads to blurry object transitions (e.g. the transition from the woman's hair to the horse in the background). This is fatal for semantic segmentation, since border pixels can not be classified well anymore, leading to poor results. When changing the aspect ratio to 1:1, such effects are stronger for dimensions that are reduced stronger (e.g. the horizontal dimension in Figure 13 because it contains more pixels).

Using higher dimensions for the input keeps finer details of objects, especially at borders. This leads to better performance during semantic segmentation because pixels can be assigned easily to objects. Increasing image dimensions leads to a similar effect as reducing pixels but not as strong as that of reduction. The system needs to create some pixels more according to interpolation rules which may lead to misleading border pixels. Still, higher input resolutions usually facilitate semantic segmentation but leading to higher computational cost.